

From a paper algorithm to Python

Project: Python readiness portfolio

1. Your mission

Your learning target

I can translate a short paper calculation into a Python script.

I can predict and explain the order of executed statements.

I can keep expected and observed output separate when checking a program.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Four packs contain six counters each. What steps find and show the total?

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Translate a short paper calculation into a Python script

Predict and explain the order of executed statements

Keep expected and observed output separate when checking a program

Retrieve what you already know

Four packs contain six counters each. What steps find and show the total?

My starting explanation

This week's end goal

Trace from a paper algorithm to python and explain the changed program before running it.

2. Notice the evidence

In Grade 6 you traced rules on paper. Now you will write a small Python program and check what it actually does. Begin with a counter-packing calculation, not the whole activity adviser. The requirement is to find the total counters in equal packs and display that total. For this first program, the supplied quantities are whole, nonnegative numbers; it does not yet validate typed input.

Use a teacher-prepared Python 3 editor that can run a saved .py file. Open a new file called counters.py, type the code exactly as shown in the next section, save it, and use the editor's Run command. Read the output area separately from the source editor. No account, AI tool, package installation or personal data is needed. If a device is unavailable, trace on paper and label the result predicted rather than executed.

List the given facts and one fact that is not supplied.

3. See how it works

A calculation from paper to output

Paper step · Python instruction · Effect

Paper step: Record pack count

Python instruction: packs = 4

Effect: Name holds 4

Paper step: Multiply two quantities

Python instruction: total = packs * per_pack

Effect: Name holds product

Paper step: Show the answer

Python instruction: print(total)

Effect: Displays the value

Assign `per_pack` before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

```
packs = 4  
per_pack = 6  
total = packs * per_pack  
print(total)
```

Read the script from top to bottom. The equals sign assigns the value on its right to the name on its left. The star means multiplication. `print(total)` displays the value of `total`; it does not display the letters in the name. Type ordinary straight characters and keep the parentheses. The Python prompt symbols `>>>` are not part of this saved script.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual output, any error and what changed. If working on paper, label both results as traces.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Change `per_pack` to 5 while keeping both `tray` inputs fixed.

Prediction: The printed total becomes 29.

Reason: 4×6 remains 24; adding five gives 29.

5. Words that unlock the idea

Word · Meaning

Word: Algorithm

Meaning: An ordered procedure for solving a stated problem

Word: Program

Meaning: Instructions written in a language a computer can execute

Word: Interpreter

Meaning: Software that executes Python instructions

Word: Assignment

Meaning: Binding a name to the value of an expression

Word: Output

Meaning: Information a program displays or returns

Use two terms to explain a step in this week's example.

6. Follow a worked example

After line · Names and values · Displayed output

After line: packs = 4

Names and values: packs is 4

Displayed output: Nothing yet

After line: `per_pack = 6`

Names and values: `packs` is 4; `per_pack` is 6

Displayed output: Nothing yet

After line: `total = packs * per_pack`

Names and values: `total` is 24

Displayed output: Nothing yet

After line: `print(total)`

Names and values: Values are unchanged

Displayed output: 24

Write 24 as the expected output before running. Then copy the actual output into a separate column. A matching result supports this case, not every possible input. A saved file may contain an old edit: save again and rerun the whole script. Do not type the expected number into the output record without running.

Why is there no displayed output after the third line? What changes if `print(total)` is placed first in a fresh run, before any assignment? Explain using which names exist.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: From a paper algorithm to Python

Convert a seed-count algorithm. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

The facts we will use

Python 3: trays=4; per_tray=6; spare=2; total=trays*per_tray+spare; print(total). All quantities count seeds.

AI ACADEMY / PYTHON-BRIDGE/1

From a paper algorithm to Python

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Name the inputs

Four trays hold six seeds each; two extra seeds are separate.



2. Translate the plan

Multiply tray count by seeds per tray, then add the spare count.



3. Trace the assignment

$4 \times 6 + 2$ is 26; total stores 26 and print displays it.

CHANGE ONE THING

Change spare to 5 while keeping both tray inputs fixed.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Name the inputs

What happens and why: Four trays hold six seeds each; two extra seeds are separate.

Step: 2. Translate the plan

What happens and why: Multiply tray count by seeds per tray, then add the spare count.

Step: 3. Trace the assignment

What happens and why: $4 \times 6 + 2$ is 26; total stores 26 and print displays it.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

What this example does not establish

The calculation assumes identical tray sizes and measured counts; it cannot detect a miscounted tray.

Read the labelled picture: Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

Change one thing: Change spare to 5 while keeping both tray inputs fixed. Predict the result before checking the explanation. What remains the same?

7. Find and repair a mistake

Change only packs to 5. Predict the result from five groups of six, save, then run the entire file again. The expected output is 30. Keep the original four-pack result beside the new one and label the changed input. A comment can explain an instruction: Python ignores text after # outside a string. A comment does not change the calculation.

```
packs = 5 # five equal packs
per_pack = 6
total = packs * per_pack
print(total)
```

A program can run successfully and still solve the wrong problem. Replacing * with + would display 11 for this input, but addition does not count five groups of six. Use the requirement and a paper calculation to check meaning; the absence of an error message is not proof of correctness.

Locate the earliest unsupported step

Move print(total) to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

State one repair and the checks you would rerun.

8. Practise together

Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line displays information?

Why is there no displayed output after the third line? What changes if `print(total)` is placed first in a fresh run, before any assignment? Explain using which names exist.

Explain your prediction to a partner before comparing results.

Practise with less help: Predict then observe

Use workbook W2 for the connected example “From a paper algorithm to Python”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for From a paper algorithm to Python. Proposed explanation: “An equals sign displays the answer”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual output, any error and what changed. If working on paper, label both results as traces.

```
packs = 4
per_pack = 6
total = packs * per_pack
print(total)
```

```
packs = 5 # five equal packs  
per_pack = 6  
total = packs * per_pack  
print(total)
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line

displays information?

Move `print(total)` to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

Explain the difference between a successful run and a correct calculation. State one limit of this program and what you would keep so another person can check your work.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled from a paper algorithm to python evidence sheet to Python readiness portfolio. Show how you can translate a short paper calculation into a python script. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

12. Show what you know

Save the file with the input values, expected and observed outputs, and a short explanation. This program follows a written multiplication rule. It has not trained a model, predicted somebody's preferences or checked that real packs contain the stated number. Its conclusion depends on the supplied data. Week 2 examines the values and types behind these names.

A script runs without an error and displays 11 for five packs of six. Is it correct for the stated requirement? Give the required result, the likely wrong operation and one check after repair. State whether your evidence came from a device run or a paper trace.

Explain the difference between a successful run and a correct calculation. State one limit of this program and what you would keep so another person can check your work.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

A calculation from paper to output

Paper step · Python instruction · Effect

Paper step: Record pack count

Python instruction: packs = 4

Effect: Name holds 4

Paper step: Multiply two quantities

Python instruction: total = packs * per_pack

Effect: Name holds product

Paper step: Show the answer

Python instruction: print(total)

Effect: Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Trace variables and a change

Predict the printed value. Change only packs to 5 and predict again. Explain which statements apply a written rule.

```
packs = 3  
  
per_pack = 7
```

```
total = packs * per_pack  
  
print(total)
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

A fictional shelf has 3 trays of 7 tiles each. Write a four-line Python script using trays, per_tray and tiles. Predict its output. Then change only trays to 0 and predict again. Explain why adding the two quantities is wrong.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a

model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.

15. Your lesson route

Start with this question: Convert a seed-count algorithm. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

By the end, I can · How I will show it

By the end, I can: Translate a short paper calculation into a Python script

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Predict and explain the order of executed statements

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Keep expected and observed output separate when checking a program

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project evidence. Your teacher may spread this lesson across several classes.

Keep your first prediction. Compare it with what the evidence shows and explain any change.

16. Finish and return

Close your notes. Explain this goal in your own words: Translate a short paper calculation into a Python script

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an input or

assumption and justify your prediction.

Save your workbook evidence. At the next review, try a different case before opening your old answer.

17. Supported practice, repair and portfolio

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Convert a seed-count algorithm. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? $4*6+2$ is 26; total stores 26 and print displays it.

1. Trace the assignment
2. Name the inputs
3. Translate the plan

My selected conclusion, deciding fact and explanation

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

After your attempt, compare with the supported explanation: Trace the assignment $4*6+2$ is 26; total stores 26 and print displays it. Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

Practice 2: explain the deciding evidence

Changed-case practice: Change spare to 5 while keeping both tray inputs fixed. Which conclusion is supported by the supplied facts?

1. The printed total becomes 29.
2. We can decide the result without examining the changed input or the procedure.
3. This one example guarantees correct results for every future input.

My selected conclusion, deciding fact and explanation

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

After your attempt, compare with the supported explanation: The printed total becomes 29. $4 \cdot 6$ remains 24; adding five gives 29.

Repair a missing building block

Review Debugging Strategies (ai-2/6). Goal: Design revealing tests for a function. Short practice: Which test most directly isolates the handling of loose books, and what should it produce?. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual output or error, change one input, then run again. Keep code, predictions, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run evidence. Open the HTML copy to read or print it. Drafts stay in this browser when storage is available; clear drafts on a shared device.